



DentiBot

Gestão odontológica acessível

DOCUMENTO 01 | ENGENHARIA E ARQUITETURA

Melhorias de Stack Tecnológico

Do protótipo acadêmico à plataforma SaaS de saúde pronta para produção — 61 melhorias priorizadas de arquitetura, dados, segurança e operação.

Projeto	DentiBot — Sistema de Gestão Odontológica
Repositório	github.com/gustavo-almeidalopes/dentibot
Branch de trabalho	claudes/stacks-improvements-business-plan-7gz5q8
Responsável	Gustavo Roberto de Almeida Lopes (líder de projeto)
Equipe	B. L. T. Chuma, M. Balsi Filho, S. S. Ferraz, G. R. A. Lopes
Versão / Data	1.0 — 07/09/2026
Classificação	Uso interno — documento de planejamento

Resumo executivo

O DentiBot já nasceu com decisões arquiteturais acima da média para um projeto acadêmico: sete microserviços Flask isolados por domínio, um API Gateway atuando como *Policy Enforcement Point*, PostgreSQL 16 único com *Row Level Security* multi-tenant (ADR-0001), Redis para *rate limiting*, balanceador Nginx à frente do gateway e um pipeline de segurança *shift-left* com gitleaks, pip-audit, SBOM CycloneDX, Bandit, Semgrep e OWASP ZAP. Esse conjunto é a base correta.

Este documento não propõe reescrever essa base — propõe fechar a distância entre o que está **declarado** e o que está **implementado**, e depois elevar a plataforma ao patamar exigido por um sistema que processa dados pessoais sensíveis de saúde sob a LGPD. A auditoria do repositório encontrou lacunas concretas e verificáveis: o docker-compose.yml anuncia *rate limiting* com Flask-Limiter e *blocklist* de JTI no Redis, mas nenhum dos sete serviços declara a dependência; os sete requirements.txt carregam PyMySQL enquanto o banco é PostgreSQL; o gateway sobe com app.run(), o servidor de desenvolvimento do Flask; o CORS está aberto para qualquer origem com supports_credentials=True; o frontend/nginx.conf escuta apenas na porta 80 embora o compose publique 443; e o repositório não contém um único teste automatizado.

As 61 melhorias abaixo estão organizadas em nove frentes e três ondas de execução. A Onda 1 (fundação, 6 semanas) elimina os riscos que hoje impediriam qualquer piloto com paciente real. A Onda 2 (escala, 12 semanas) prepara a plataforma para multi-clínica com observabilidade e entrega contínua. A Onda 3 (plataforma, 24 semanas) constrói a camada de dados e IA que sustenta as funcionalidades do Documento 03. Cada item traz o arquivo ou serviço alvo, uma estimativa de esforço e um indicador objetivo de sucesso.

61	7	1.565	0	3
melhorias mapeadas	microserviços auditados	linhas Python analisadas	testes automatizados hoje	ondas de execução

Sumário

01	Diagnóstico do estado atual
02	Camada de aplicação e runtime
03	Dados e persistência
04	Comunicação entre serviços e resiliência
05	Observabilidade e confiabilidade
06	Segurança de plataforma e cadeia de suprimentos
07	Containers, orquestração e infraestrutura
08	Qualidade, testes e entrega contínua
09	Frontend e aplicativos móveis
10	Plataforma de dados e IA
11	Anexos
Anexo A — Stack atual versus stack alvo	
Anexo B — Roadmap por ondas de execução	
Anexo C — Riscos de execução e mitigação	

01

Diagnóstico do estado atual

O que a leitura do repositório mostra antes de qualquer proposta

O diagnóstico abaixo é factual: cada linha aponta uma divergência entre a intenção declarada na documentação ou no docker-compose.yml e o que o código efetivamente faz hoje. Essas divergências não são defeitos de concepção — são o resultado natural de um projeto que evoluiu a documentação de arquitetura mais rápido que a implementação. Corrigi-las é o caminho mais barato para ganho de qualidade, porque o desenho já está certo.

Componente	Situação encontrada no repositório	Consequência prática
Gateway (gateway/app.py)	Sobe com app.run() do Flask; proxy síncrono com requests e timeout fixo de 10 s; sem retry, sem circuit breaker.	Um serviço lento bloqueia workers do gateway e derruba a aplicação inteira.
Dependências dos serviços	Os sete requirements.txt declaram PyMySQL, mas o ADR-0001 define PostgreSQL como banco único.	Driver morto no SBOM, superfície de ataque desnecessária e sinal de deriva documentação/código.
Rate limiting	O compose comenta 'Redis — rate limiting (Flask-Limiter) + blocklist de JTI'; Flask-Limiter não aparece em nenhum requirements.txt.	As rotas de login e 2FA aceitam tentativas ilimitadas: força bruta e enumeração de usuários.
CORS	CORS(app, supports_credentials=True) sem lista de origens, tanto no gateway quanto nos serviços.	Qualquer site pode emitir requisições autenticadas em nome do usuário.
TLS	O compose publica 80 e 443, mas frontend/nginx.conf declara apenas listen 80.	Tráfego com prontuário e token JWT pode trafegar em claro.
Sessão / JWT	Token HS256 validado no gateway; sem blocklist de JTI, sem refresh token, guardado em localStorage por src/js/api.js.	Logout não invalida sessão e um XSS exfiltra o token diretamente.
Banco de dados	Schema aplicado por scripts em /docker-entrypoint-initdb.d; sem ferramenta de migração; políticas RLS existem mas o app não define o tenant na sessão.	Não há evolução de schema em produção nem isolamento multi-tenant efetivo.
Testes	Nenhum arquivo de teste, nenhuma configuração de pytest, nenhum gate de cobertura no ci-security.	Toda alteração é validada manualmente; regressões silenciosas em prontuário.
Observabilidade	Sem logs estruturados, métricas, tracing ou alertas. /health do gateway consulta os sete serviços a cada chamada.	Incidente é descoberto pelo cliente; o health check vira amplificador de carga.
Frontend	40+ páginas HTML estáticas com marcação repetida; sem etapa de build, bundling ou versionamento de assets.	Manutenção cara e cache difícil de invalidar em atualização.

O que já está certo e deve ser preservado

A separação em sete domínios (auth, patient, appointment, inventory, communication, financial, audit) espelha os limites de negócio corretos e deve ser mantida. A decisão de consolidar em um único PostgreSQL com RLS em vez de sete bancos MySQL (ADR-0001) é acertada para o estágio atual. O uso de papéis de banco com privilégio mínimo (dentibot_app, dentibot_tokenizer, dentibot_replicator), validade de senha de 90 dias, cofre de tokenização em schema separado, pgaudit e pg_cron coloca o projeto acima da média do mercado. O pipeline ci-security com gates bloqueantes é exemplar. As melhorias a seguir constroem sobre isso, não contra isso.

02

Camada de aplicação e runtime

Como os sete serviços Flask são executados, configurados e padronizados

Como ler as tabelas de melhorias

ID identifica a melhoria de forma estável e pode ser referenciada em issues, commits e pull requests. **Onde se aplica** aponta o arquivo, serviço ou área do repositório afetada. **Impacto** estima o ganho percebido por usuário ou negócio (Alto / Médio / Baixo). **Esf.** é o esforço estimado de implementação: **P** até 3 dias-pessoa, **M** de 4 a 15 dias-pessoa, **G** acima de 15 dias-pessoa. As estimativas são de planejamento e devem ser refinadas em refinamento técnico antes de virarem compromisso de sprint.

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-01	Servidor WSGI de produção com Gunicorn Substituir <code>app.run()</code> por Gunicorn com workers gthread ou gevent, <code>--graceful-timeout</code> , <code>--max-requests</code> com jitter para reciclar workers e evitar vazamento de memória. O servidor embutido do Flask é single-threaded e não suporta carga concorrente nem encerramento gracioso em <i>rolling deploy</i> .	gateway/, services/*/	Alto	P	Suporta 200 req/s por réplica sem erro 5xx
ST-02	Application Factory e Blueprints padronizados Os sete <code>app.py</code> repetem inicialização, CORS e registro de rotas. Adotar o padrão <code>create_app(config)</code> com Blueprints permite instanciar a aplicação com configuração de teste, isolar extensões e eliminar estado global.	services/*/app.py	Alto	M	Aplicação instanciável em teste sem efeitos colaterais
ST-03	Biblioteca interna dentibot-common Extrair para um pacote versionado o que hoje é copiado entre serviços: decorador de autenticação, modelo de erro, logger, cliente HTTP interno, leitura de configuração e helpers de tenant. Publicar em registry privado e consumir por versão fixada.	novo pacote + services/*	Alto	M	Redução de ~40% da duplicação entre serviços
ST-04	Gateway assíncrono orientado a I/O O gateway é essencialmente um proxy: seu trabalho é esperar rede. Migrar de requests síncrono para <code>httpx.AsyncClient</code> sobre FastAPI ou Quart multiplica a concorrência por worker sem aumentar hardware.	gateway/app.py	Alto	M	3× a 5× mais conexões concorrentes por réplica
ST-05	Validação e serialização com Pydantic v2 Nenhuma rota valida esquema de entrada hoje. Definir modelos de request/response por endpoint dá validação declarativa, mensagens de erro consistentes, documentação automática e proteção contra <i>mass assignment</i> .	services/*/routes.py	Alto	M	100% dos endpoints com esquema declarado
ST-06	Contrato de erro RFC 9457 (Problem Details) Padronizar todas as respostas de erro em <code>application/problem+json</code> com type, title, status, detail, instance e trace_id, substituindo o ad-hoc <code>{"erro": "..."}</code> . O frontend passa a tratar erro por código, não por texto.	gateway/, services/*	Médio	P	Erro rastreável do browser ao log por trace_id
ST-07	Configuração tipada com fail-fast em segredo ausente JWT_SECRET tem hoje o valor padrão "fallback-secret": um deploy sem variável de ambiente sobe com segredo público e assina tokens válidos. Trocar por pydantic-settings que recusa iniciar sem os segredos obrigatórios.	gateway/app.py:15	Alto	P	Zero segredo com valor padrão no código

03

Dados e persistência

PostgreSQL, migrações, isolamento multi-tenant e continuidade

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-08	Remover PyMySQL de todos os requirements O banco é PostgreSQL desde o ADR-0001, mas seis serviços ainda instalam PyMySQL==1.1.0. É dependência morta que entra no SBOM CycloneDX, aumenta a superfície do pip-audit e induz erro em quem lê o projeto pela primeira vez.	services/*/requirements.txt	Médio	P	SBOM sem dependência não utilizada
ST-09	psycopg 3 com pool e PgBouncer Adotar psycopg[binary,pool] e colocar PgBouncer em modo transaction entre aplicação e banco. O papel dentibot_app tem CONNECTION LIMIT 80: sem pool, escalar o gateway para três réplicas esgota o limite e derruba serviços.	services/*/docker-compose.yml	Alto	M	Conexões ativas < 40% do limite sob pico
ST-10	Migrações versionadas com Alembic O schema é aplicado uma única vez por /docker-entypoint-initdb.d, o que só funciona em banco vazio. Sem migração versionada não existe evolução de schema em produção, nem rollback, nem histórico de mudança auditável.	database/, novo alembic/	Alto	M	Deploy de schema reversível e auditável
ST-11	Ativar RLS de fato propagando o tenant As políticas de <i>Row Level Security</i> existem no schema, mas só entram em vigor se a aplicação executar SET LOCAL app.current_clinica a cada transação. Implementar isso no middleware, a partir do claim do JWT, e testar o vazamento entre clínicas.	dentibot-common, services/*	Alto	M	Teste automatizado provando isolamento entre tenants
ST-12	Roteamento efetivo leitura/escrita Os arquivos db_leitura.sql e replicacao.sql preveem réplica, mas todos os serviços apontam para a mesma DATABASE_URL. Configurar réplica física, roteador de sessão e tolerância a atraso de replicação em relatórios.	database/, docker-compose.yml	Médio	M	60% das leituras servidas pela réplica
ST-13	Observabilidade e tuning de consultas Habilitar pg_stat_statements e auto_explain, revisar índices das consultas de agenda (faixa de data por clínica e por profissional) e criar orçamento de latência por consulta no painel do banco.	docker/postgres/	Médio	P	p95 de consulta de agenda < 50 ms
ST-14	Backup PITR com restauração testada O SECURITY.md exige backups diários; falta a implementação e, principalmente, o teste. Adotar pgBackRest ou WAL-G com <i>point-in-time recovery</i> e um job mensal que restaura em ambiente isolado e valida integridade — backup não testado não é backup.	infra, docker/postgres/	Alto	M	RPO ≤ 5 min, RTO ≤ 1 h, restauração testada/mês
ST-15	Particionamento e política de retenção Tabelas de auditoria e de logs de comunicação crescem indefinidamente. Particionar por mês com pg_cron (já carregado) e definir retenção alinhada à finalidade declarada na LGPD, com expurgo ou anonimização automática.	database/04-audit-extensions.sql	Médio	M	Custo de armazenamento estável ano a ano

04

Comunicação entre serviços e resiliência

Como o sistema se comporta quando uma parte dele falha

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-16	Timeouts, retries com jitter e circuit breaker O gateway usa timeout fixo de 10 s e nenhuma política de retry. Definir timeout por rota (conexão e leitura separados), retry com <i>exponential backoff</i> e jitter apenas para métodos idempotentes, e disjuntor que abre após N falhas para não propagar a queda.	gateway/app.py:66-88	Alto	M	Falha de um serviço não derruba o gateway
ST-17	Autenticar de fato o tráfego east-west INTERNAL_SERVICE_SECRET é injetado em todos os serviços pelo compose, mas nenhum código o utiliza: hoje qualquer processo na rede Docker chama qualquer serviço interno. Implementar assinatura HMAC dos cabeçalhos ou mTLS entre serviços.	gateway/, services/*	Alto	M	Chamada interna sem assinatura válida é rejeitada
ST-18	Barramento de eventos assíncrono Notificação, auditoria e cobrança não precisam ser síncronas. Publicar eventos de domínio (consulta.agendada, pagamento.confirmado) em Redis Streams e, com o volume, migrar para RabbitMQ ou Kafka gerenciado.	novo barramento, services/*	Alto	G	Latência de agendamento reduzida em ~30%
ST-19	Outbox transacional e idempotência Garantir que um agendamento gravado sempre gere sua notificação, mesmo com falha de rede, via padrão <i>outbox</i> . Exigir cabeçalho Idempotency-Key em POST de agendamento e cobrança para eliminar duplicidade por reenvio.	appointment-service, financial-service	Alto	M	Zero cobrança ou consulta duplicada
ST-20	Contratos OpenAPI 3.1 e testes de contrato Gerar especificação a partir dos modelos Pydantic (ST-05), publicá-la como artefato do CI e rodar testes de contrato com Schemathesis. Isso trava quebras de compatibilidade antes que cheguem ao frontend e aos apps móveis.	services/*, .github/workflows/	Alto	M	Quebra de contrato detectada no PR
ST-21	Gateway maduro no lugar do proxy manual As sete funções de rota do gateway repetem a mesma lógica de encaminhamento. Migrar para Kong, Traefik ou Envoy entrega roteamento declarativo, rate limiting, autenticação, retries e métricas sem código próprio para manter.	gateway/, docker-compose.yml	Médio	G	~70% menos código próprio de roteamento
ST-22	Limite de corpo e resposta em streaming request.get_data() carrega o corpo inteiro em memória antes de repassar: um upload grande de imagem radiográfica derruba o gateway. Aplicar MAX_CONTENT_LENGTH, encaminhamento em streaming e upload direto ao object storage com URL pré-assinada.	gateway/app.py:74-80	Alto	M	Upload de 50 MB sem crescimento de memória

05

Observabilidade e confiabilidade

Enxergar o sistema antes que o cliente enxergue o problema

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-23	Instrumentação OpenTelemetry ponta a ponta Adotar OTel para traces, métricas e logs, com propagação de traceparent do browser ao banco. Em uma malha de sete serviços, sem tracing distribuído a investigação de lentidão é adivinhação.	gateway/, services/*	Alto	M	100% das requisições com trace correlacionado
ST-24	Logs estruturados com mascaramento de PII Emitir JSON com trace_id, clinica_id, user_id e rota, e um filtro que mascara CPF, telefone, e-mail e conteúdo clínico antes da escrita. Log com dado de saúde em claro é incidente de LGPD por si só.	dentibot-common	Alto	M	Zero PII em amostragem de log auditada
ST-25	Prometheus, Grafana, SLO e error budget Expor métricas RED (taxa, erro, duração) por serviço e USE para infraestrutura; publicar SLOs (99,5% de disponibilidade, p95 < 400 ms na agenda) e governar releases pelo orçamento de erro consumido.	infra, services/*	Alto	M	SLO publicado e medido continuamente
ST-26	Separar liveness de readiness O /health do gateway consulta os sete serviços a cada chamada: sob monitoramento agressivo ele mesmo vira carga (problema N+1). Separar /health/live (barato, local) de /health/ready (dependências, com cache curto) e adicionar healthcheck aos serviços no compose.	gateway/app.py:91-101	Alto	P	Health check com custo constante e previsível
ST-27	Alertas acionáveis e runbooks Configurar Alertmanager com alertas baseados em sintoma (erro de usuário, latência, fila) em vez de causa, cada um ligado a um runbook versionado no repositório. Alerta sem runbook vira ruído e é ignorado.	infra, docs/runbooks/	Médio	M	MTTR < 30 min nos incidentes críticos
ST-28	Rastreamento de exceções com contexto Integrar Sentry ou GlitchTip com versão de release, ambiente e usuário pseudonimizado; agrupar por <i>fingerprint</i> e falhar o deploy quando a taxa de novas exceções ultrapassar o limite definido.	gateway/, services/*, src/js/	Médio	P	Regressão detectada em < 5 min do deploy

06

Segurança de plataforma e cadeia de suprimentos

Fechar a distância entre o SECURITY.md e o código executado

O SECURITY.md do projeto descreve dez padrões de segurança com controles exigidos e implementação esperada — é um documento sólido. Os itens desta seção tratam especificamente dos controles que estão descritos lá mas ainda não aparecem no código, além de reforços de cadeia de suprimentos que um SaaS de saúde precisa demonstrar em auditoria de cliente corporativo.

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-29	Rate limiting real nas rotas sensíveis Instalar Flask-Limiter com backend Redis (já provisionado) e aplicar limites por IP e por conta em /auth/login, /auth/verify-2fa e /auth/register, com bloqueio progressivo. Hoje não há qualquer limite: força bruta contra 2FA de 6 dígitos é viável.	services/auth-service/, gateway/	Alto	P	Força bruta bloqueada em < 10 tentativas
ST-30	CORS restrito por origem declarada Trocar CORS(app, supports_credentials=True) por lista explícita de origens lida de FRONTEND_ORIGIN (variável que já existe no compose, mas não é usada no código), com métodos e cabeçalhos permitidos enumerados.	gateway/app.py:14	Alto	P	Origem não autorizada recebe 403 no preflight
ST-31	Ciclo de vida de sessão: refresh, JTI e revogação Emitir access token curto (15 min) com jti, refresh token rotativo com detecção de reuso e blocklist de JTI em Redis. Hoje o logout apenas apaga o token no browser — o token continua válido até expirar.	services/auth-service/routes.py	Alto	M	Revogação efetiva em < 60 s
ST-32	Cabeçalhos de segurança e CSP frontend/nginx.conf não define nenhum cabeçalho de segurança. Adicionar CSP restritiva, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy, Permissions-Policy e frame-ancestors.	frontend/nginx.conf	Alto	P	Nota A na avaliação do Mozilla Observatory
ST-33	TLS efetivo e redirecionamento forçado O compose publica a porta 443 mas o servidor Nginx só declara listen 80. Configurar TLS 1.3 com certificado automatizado (ACME/Let's Encrypt), redirecionamento 301 de 80 para 443 e renovação automática monitorada.	frontend/nginx.conf, docker/	Alto	P	Zero tráfego de aplicação em texto claro
ST-34	Gestão e rotação automática de segredos Os papéis do PostgreSQL já expiram em 90 dias (VALID UNTIL), o que quebrará a aplicação sem um processo de rotação. Adotar Vault ou SOPS com rotação automatizada e recarga sem downtime, eliminando segredo em variável de ambiente de longa duração.	infra, docker-compose.yml	Alto	M	Rotação sem indisponibilidade a cada 90 dias
ST-35	Proveniência: assinatura de imagem e SLSA O CI já gera SBOM CycloneDX. Completar com assinatura das imagens via cosign, atestado de proveniência SLSA nível 2 e verificação obrigatória na admissão do cluster — imagem não assinada não sobe.	.github/workflows/ci.yml	Médio	M	100% das imagens assinadas e verificadas

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-36	Programa de gestão de vulnerabilidades O Dependabot já está configurado. Formalizar SLA de correção por severidade (crítica em 72 h, alta em 7 dias), varredura de imagem com Trivy ou Gype além do pip-audit, e publicar um security.txt com canal de divulgação responsável.	.github/, frontend/	Médio	P	Zero CVE crítica aberta há mais de 72 h

07

Containers, orquestração e infraestrutura

Do docker-compose de estudo à infraestrutura reproduzível

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-37	Dockerfile endurecido e multi-stage Os Dockerfiles copiam o projeto inteiro e executam como root sobre python:3.11-slim sem digest fixo. Adotar build multi-stage, usuário não-root, base pinada por digest, .dockerignore, HEALTHCHECK e sistema de arquivos somente leitura.	gateway/Dockerfile, services/*/Dockerfile	Alto	P	Imagem < 150 MB rodando sem root
ST-38	Higienizar o docker-compose Remover a chave version: '3.8' (obsoleta no Compose V2), declarar limites de CPU e memória por serviço, mover a exposição de 127.0.0.1:5432 para um profile de desenvolvimento e adicionar healthcheck aos sete serviços de aplicação.	docker-compose.yml	Médio	P	Compose sem aviso e com limites declarados
ST-39	Orquestração com autoscaling O comentário do compose sugere escalar com --scale, o que é manual e sem reação a carga. Migrar para Kubernetes gerenciado (EKS/GKE/OKE) com HPA por métrica de requisição, PodDisruptionBudget e rolling update sem downtime.	infra, docker-compose.yml	Alto	G	Escala automática entre 2 e 10 réplicas
ST-40	Infraestrutura como código Descrever rede, banco, cache, cluster, DNS e certificados em Terraform, com estado remoto e ambientes <i>dev</i> , <i>homologação</i> e <i>produção</i> isolados. Hoje a infraestrutura só existe no compose de desenvolvimento.	novo infra/terraform/	Alto	G	Ambiente recriado do zero em < 1 h
ST-41	Armazenamento de objetos para anexos clínicos Radiografias, fotos intraorais e documentos assinados não devem trafegar pelo banco nem pelo gateway. Usar S3 compatível com criptografia no repouso, URL pré-assinada, versionamento e <i>object lock</i> para o que exige retenção legal.	patient-service, infra	Alto	M	Anexos servidos sem passar pelo gateway
ST-42	CDN e otimização de entrega estática O Nginx já aplica gzip e cache de 30 dias. Completar com CDN, Brotli, <i>fingerprint</i> no nome do arquivo para <i>cache busting</i> seguro e HTTP/2 ou HTTP/3 no <i>edge</i> .	frontend/nginx.conf	Médio	P	LCP < 2,0 s no 4G brasileiro
ST-43	Plano de continuidade e recuperação de desastre Definir RTO e RPO por serviço, distribuir em múltiplas zonas de disponibilidade, documentar o procedimento de failover e ensaiá-lo em <i>game day</i> semestral. Clínica parada é receita perdida no dia.	infra, docs/runbooks/	Alto	M	Failover ensaiado com RTO ≤ 1 h

08

Qualidade, testes e entrega contínua

O maior risco isolado do projeto hoje

Risco número um: ausência total de testes automatizados

A varredura do repositório não encontrou nenhum arquivo de teste, nenhuma configuração de pytest e nenhum gate de cobertura no workflow ci-security. O pipeline verifica segurança (gitleaks, pip-audit, Bandit, Sengrep, ZAP) com rigor, mas não verifica **comportamento**. Em um sistema que grava prontuário odontológico, agenda e cobrança, uma regressão silenciosa em regra de negócio é mais provável e mais cara que uma CVE de dependência. Nenhuma das outras 60 melhorias deste documento pode ser executada com segurança sem uma rede de testes que prove que nada quebrou.

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-44	Suíte de testes automatizados com gate de cobertura Estruturar pytest com pirâmide de testes: unitários de regra de negócio, integração com PostgreSQL efêmero via testcontainers e testes de API por serviço. Exigir cobertura mínima de 70% nas linhas alteradas como gate bloqueante de PR.	novo tests/, .github/workflows/ci.yml	Alto	G	Cobertura ≥ 70% e gate ativo no PR
ST-45	Lint, formatação e tipagem no CI Adotar ruff (lint e formatação) e mypy em modo estrito incremental, executados por pre-commit localmente e como job do CI. Reduz revisão de estilo e captura erro de tipo antes da execução.	.github/workflows/, .pre-commit-config.yaml	Médio	P	Zero achado de lint no main
ST-46	Testes end-to-end dos fluxos críticos Automatizar com Playwright os caminhos que não podem quebrar: cadastro, login com 2FA, seleção de perfil, agendamento, registro de prontuário e cobrança. Executar contra ambiente efêmero a cada PR e no <i>smoke test</i> pós-deploy.	novo e2e/, src/pages/	Alto	M	6 jornadas críticas cobertas em cada PR
ST-47	Testes de carga com meta de latência Modelar carga realista (pico de agendamento entre 8h e 10h, envio de lembretes em lote) com k6, definir metas de p95 por endpoint e falhar o pipeline de release quando regredir mais de 20% em relação à linha de base.	novo perf/, .github/workflows/	Médio	M	p95 da agenda < 400 ms com 500 usuários
ST-48	Pipeline de build, publicação e deploy O CI hoje só verifica; não constrói nem publica imagens. Criar workflow que constrói com cache de camadas, publica em registry versionado por SHA, faz deploy automático em homologação e exige aprovação para produção, com rollback em um comando.	.github/workflows/	Alto	M	Lead time de commit a produção < 1 dia
ST-49	Ambientes efêmeros por pull request Provisionar um ambiente descartável por PR, com dados sintéticos, para revisão funcional e execução dos testes E2E e do ZAP baseline (que hoje só roda semanalmente contra homologação) antes do merge.	.github/workflows/dast.yml	Médio	G	DAST executado em 100% dos PRs de frontend

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-50	Versionamento semântico e histórico legível Adotar Conventional Commits, semantic-release e CHANGELOG gerado. O histórico atual ("Add files via upload" repetido) impede rastrear quando e por que uma mudança entrou — informação exigida em auditoria de software de saúde.	repositório, .github/	Médio	P	100% dos releases com changelog rastreável

09

Frontend e aplicativos móveis

Da entrega estática duplicada a uma base de código sustentável

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-51	Token fora do localStorage src/js/api.js guarda o JWT em localStorage, acessível a qualquer script da página: um único XSS exfiltra a sessão de um sistema de prontuário. Migrar para cookie httpOnly, Secure, SameSite=Strict com proteção CSRF por token duplo.	src/js/api.js:11-25	Alto	M	Token inacessível a JavaScript
ST-52	Etapa de build com Vite e TypeScript Introduzir build moderno mantendo a entrega estática pelo Nginx: TypeScript para o cliente de API, <i>bundling</i> , <i>minificação</i> , <i>tree shaking</i> e <i>hash</i> no nome do arquivo para cache seguro.	src/js/, frontend/	Médio	M	Bundle inicial < 100 KB comprimido
ST-53	Eliminar duplicação de marcação Existem 40+ páginas HTML com cabeçalho, rodapé e navegação repetidos, e src/partials/*.php que o Nginx estático nunca processa. Gerar as páginas a partir de <i>templates</i> em tempo de build ou adotar Web Components.	src/pages/, src/partials/	Alto	M	Alteração de cabeçalho em 1 arquivo, não em 40
ST-54	Progressive Web App com modo offline Consultório perde conexão. Um <i>service worker</i> com cache da agenda do dia e fila de sincronização permite consultar e registrar atendimento offline, sincronizando ao reconectar — resolve uma dor real do público-alvo.	src/, novo sw.js	Alto	M	Agenda do dia utilizável sem conexão
ST-55	Cliente de API gerado a partir do OpenAPI Com o contrato de ST-20, gerar clientes tipados para web, Kotlin e Swift. Elimina a reescrita manual em três linguagens e garante que os apps quebrem em tempo de compilação, não em produção.	src/js/api.js, mobile/	Médio	M	Três clientes sempre sincronizados com a API
ST-56	Aplicativos móveis conectados à API real mobile/android/index.kt e mobile/ios/index.swift reproduzem a landing page em Compose e SwiftUI, sem consumir a API. Definir escopo do app (agenda, prontuário de consulta, confirmação do paciente) e avaliar Kotlin Multiplatform para compartilhar a camada de domínio.	mobile/	Alto	G	App em beta fechado com fluxo de agenda funcional

10

Plataforma de dados e IA

A fundação técnica exigida pelas funcionalidades do Documento 03

As funcionalidades propostas no Documento 03 — copiloto clínico, previsão de faltas, triagem por imagem, transcrição de atendimento — não são *features* que se instalam: cada uma depende de uma camada de dados governada que ainda não existe. Os itens a seguir são pré-requisitos técnicos, não opcionais, e devem estar concluídos antes do primeiro modelo entrar em produção.

ID	Melhoria proposta	Onde se aplica	Impacto	Esf.	Indicador de sucesso
ST-57	Separar carga analítica da transacional Relatórios gerenciais executados sobre o banco de produção competem com o atendimento. Criar camada analítica alimentada por CDC com modelagem em dbt, servindo o painel gerencial e os modelos preditivos sem tocar o OLTP.	novo analytics/ financial-service	Alto	G	Zero consulta de relatório no banco OLTP
ST-58	Catálogo de dados e feature store Registrar origem, significado, dono e base legal de cada atributo, e materializar as <i>features</i> dos modelos (histórico de faltas, intervalo entre consultas, sazonalidade) com a mesma definição em treino e em inferência.	novo analytics/	Médio	G	Zero divergência treino/produção nas features
ST-59	pgvector para busca semântica Habilitar a extensão pgvector no PostgreSQL existente para busca semântica sobre protocolos clínicos, notas de evolução e base de conhecimento — a fundação do RAG sem introduzir um banco vetorial novo na operação.	docker/postgres/ database/	Médio	M	Busca semântica com p95 < 200 ms
ST-60	Pipeline de anonimização para uso secundário Uso de dado clínico para treinar modelo exige base legal distinta do atendimento. Construir pipeline de pseudonimização e anonimização (k-anonimato, supressão de identificadores diretos) reaproveitando o cofre de tokenização já previsto no schema.	database/05-tokenization-e xtensions.sql	Alto	G	Conjunto de treino sem dado identificável
ST-61	Gateway de LLM com governança Centralizar todo acesso a modelos de linguagem em um serviço próprio com cota por clínica, cache semântico, <i>fallback</i> entre provedores, redação de PII no prompt e registro auditável de entrada e saída. Sem isso o custo é imprevisível e a rastreabilidade exigida pela LGPD é impossível.	novo ai-gateway/	Alto	G	100% das chamadas de IA auditadas e com custo por clínica

11

Anexos

Stack alvo, roadmap e riscos de execução

Anexo A — Stack atual versus stack alvo

Camada	Hoje	Alvo (12 meses)
Execução	Flask app.run(), servidor de desenvolvimento	Gunicorn/Uvicorn com autoscaling em Kubernetes
Framework	Flask 2.3.3 síncrono nos 7 serviços	Flask com Blueprints nos serviços; FastAPI assíncrono no gateway
Validação	Nenhuma; JSON manipulado direto	Pydantic v2 com OpenAPI 3.1 gerado
Banco	PostgreSQL 16 + RLS declarado, PyMySQL nos requisitos	PostgreSQL 16 + RLS ativo, psycopg 3, PgBouncer, réplica de leitura
Schema	Scripts SQL no initdb, sem versionamento	Alembic com migração reversível
Cache/Fila	Redis provisionado, sem uso no código	Redis para rate limiting, blocklist de JTI, cache e Streams
Autenticação	JWT HS256 em localStorage, sem revogação	Access token curto + refresh rotativo em cookie httpOnly, blocklist de JTI
Comunicação interna	HTTP simples, segredo interno não utilizado	mTLS ou HMAC, retries, circuit breaker, eventos assíncronos
Observabilidade	Nenhuma	OpenTelemetry, Prometheus, Grafana, Sentry, SLO
Testes	Nenhum	pytest com 70% de cobertura, Playwright E2E, k6
CI/CD	ci-security (SAST, SCA, segredos) + DAST semanal	Mesmos gates + build, testes, publicação assinada e deploy contínuo
Frontend	40+ HTML estáticos, JS sem build	Vite + TypeScript, PWA offline, CDN
Mobile	Telas estáticas Compose e SwiftUI	Apps conectados à API com cliente gerado
Dados/IA	Inexistente	CDC + dbt, feature store, pgvector, gateway de LLM auditado

Anexo B — Roadmap por ondas de execução

As ondas não são sequenciais rígidas: itens de onda posterior podem ser antecipados quando desbloqueiam trabalho de negócio. A regra é que **nenhum item de onda 2 ou 3 entra antes de ST-44** (suíte de testes), porque sem rede de testes cada melhoria é também um risco de regressão.

Onda	Janela	Itens	Objetivo de negócio
Onda 1 — Fundação	Semanas 1 a 6	ST-01, ST-07, ST-08, ST-26, ST-29, ST-30, ST-32, ST-33, ST-38, ST-44, ST-45, ST-51	Eliminar os bloqueios que hoje impedem um piloto com dados de paciente real.
Onda 2 — Escala	Semanas 7 a 18	ST-02 a ST-06, ST-09 a ST-14, ST-16, ST-17, ST-19, ST-20, ST-22 a ST-25, ST-27, ST-28, ST-31, ST-34 a ST-37, ST-41, ST-42, ST-46 a ST-50, ST-52 a ST-54	Operar dezenas de clínicas com observabilidade, entrega contínua e custo previsível.
Onda 3 — Plataforma	Semanas 19 a 40	ST-15, ST-18, ST-21, ST-39, ST-40, ST-43, ST-55 a ST-61	Sustentar as funcionalidades de IA do Documento 03 e a expansão do Documento 05.

Anexo C — Riscos de execução e mitigação

Risco	Efeito se ocorrer	Mitigação
Refatorar sem rede de testes	Regressão silenciosa em prontuário ou cobrança descoberta pelo cliente.	ST-44 antes de qualquer refatoração estrutural; caracterizar comportamento atual com testes antes de alterar.
Equipe acadêmica com rotatividade	Conhecimento concentrado em uma pessoa e trabalho parado na troca de semestre.	ADRs versionados, runbooks (ST-27), pareamento e revisão obrigatória de PR.
Escopo de IA antes da fundação de dados	Modelo em produção sem base legal, sem rastreabilidade e com custo descontrolado.	ST-57 a ST-61 como pré-requisito formal de qualquer item do Documento 03.
Custo de nuvem acima do previsto	Margem do plano Solo (R\$ 97/mês) comprometida por cliente.	Limites de recurso (ST-38), painel de custo por clínica, cache semântico e cota no gateway de LLM (ST-61).
Migração de schema em produção	Indisponibilidade durante deploy ou perda de dado.	Alembic com migração <i>expand/contract</i> , ensaio em cópia mascarada e PITR testado (ST-14).
Divergência entre documentação e código	Auditoria de cliente corporativo encontra controle declarado e não implementado.	Revisão trimestral do SECURITY.md contra o código, com evidência automatizada no CI.

Método. As melhorias foram derivadas da leitura direta do repositório na branch `claude/stacks-improvements-business-plan-7gz5q8`: `docker-compose.yml`, `gateway/app.py`, os sete serviços em `services/`, `database/*.sql`, `frontend/nginx.conf`, `src/js/`, `mobile/`, `.github/workflows/` e `SECURITY.md`. Toda afirmação sobre o estado atual é verificável no código citado na coluna "Onde se aplica". As estimativas de esforço e os indicadores de sucesso são projeções de planejamento e devem ser revalidadas em refinamento técnico com a equipe. Metas numéricas (p95, cobertura, RTO) são propostas de partida, a serem ajustadas após medir a linha de base real em ambiente de homologação.